Cómo "aprende" o recuerda cosas el bot de Patata

> Escrito 2026-07-27 por Alma, para Gonza y (si hace falta compartirlo) Christian. Lenguaje simple a propósito: el objetivo es que cualquiera de los dos sepa, sin mirar código, **qué tocar si algún día quieren que el bot "aprenda" algo distinto.**

La idea en una frase

El bot no "aprende" solo, como lo haría una persona con la experiencia. Todo lo que el bot "sabe" hoy viene de **archivos de texto que alguien edita** (Christian o Alma) más un puñado de **cosas que el código recuerda por un rato y después olvida solas**. No hay ningún mecanismo de aprendizaje automático real: es más parecido a un empleado que lee un manual antes de cada turno — algunas páginas del manual son fijas, otras las reescribe el jefe cada mañana.

---

1. Lo PERMANENTE — sobrevive para siempre, hasta que alguien lo edite a mano

Esto es lo más parecido a "lo que el bot sabe de memoria". No se borra solo nunca; solo cambia si una persona lo edita.

Qué	Quién lo edita	Cómo
`conocimiento_local.md`	**Christian**, directo en GitHub	Reglas del negocio en texto libre: horarios de excepción, aclaraciones de carta, matices que Christian quiere que el bot sepa. Se manda COMPLETO a la IA en cada mensaje.
`system_prompt.md` / `system_prompt_jefe.md`	**Alma** (o Gonza pidiéndoselo a Alma)	El tono, las reglas de comportamiento, el vocabulario, cómo cerrar un pedido, qué hacer ante un reclamo. Es "la personalidad" del bot — cambiarlo es trabajo de código/sesión, no algo que Christian edite solo.
El menú (`menu.json`)	**Christian**, desde el Admin del POS (⚙ Admin → Menú)	Catálogo, precios, promos, horario, delivery, formas de pago. **Desde el 27-jul (S4 del plan) hay una sola fuente:** el bot ya NO tiene su propio menú aparte — cada 5 minutos trae el menú real del POS por el túnel y guarda una copia local (si el POS o el túnel están caídos en ese momento, sigue funcionando con la última copia buena, nunca se queda sin menú). Cambiar un precio o agregar un producto en el POS se refleja solo en el bot, sin tocar código ni pedirle nada a Alma.
El código mismo (`webhook.py`,	**Alma**, vía sesión + commit + push a	Todo lo que es lógica dura: cómo se arma la comanda, cuándo se reintenta un WhatsApp,

Qué	Quién lo edita	Cómo
`prototipo.py`)	Railway	cuándo se deriva a un humano. Cambiar esto siempre pasa por una sesión completa (probar, commitear, pushear, aprobar el deploy en Railway a mano).
`mejoras_jefe.md`	Se **escribe solo** (el jefe manda una idea y el código la anota con fecha), pero **nadie la lee automáticamente**	Cuando Christian le dice al bot algo tipo "sería bueno que...", queda anotado ahí para siempre. **Importante:** esto NO cambia el comportamiento del bot por sí solo — es una lista de pendientes para que Alma la revise y, si corresponde, la convierta en un cambio real de `conocimiento_local.md` o `system_prompt.md`.

En resumen: si quieren que el bot "aprenda" una regla de negocio nueva y que dure para siempre, hay dos caminos — Christian edita `conocimiento_local.md` él mismo, o le piden a Alma que ajuste `system_prompt.md` / el código en una sesión.

---

2. Lo TEMPORAL — se resetea solo, sin que nadie tenga que acordarse

Esto es memoria de corto plazo. Existe para que el bot no se sienta robótico dentro de una conversación o un turno de servicio, pero está diseñado a propósito para **no acumularse para siempre** — así el bot no arrastra contexto viejo ni se equivoca con información vencida.

Qué	Cuánto dura	Por qué se resetea así
`instrucciones_local.txt` (el aviso del día: "solo retiro hoy", "se agotó tal cosa")	**Hasta la medianoche** (expira apenas cambia la fecha en Chile)	Antes quedaba pegado para siempre hasta que alguien se acordaba de borrarlo a mano — pasó de verdad que un "solo retiro" quedó vigente días después. Ahora el archivo lleva la fecha en que se dictó, y si esa fecha ya no es HOY, el bot lo ignora solo. Christian lo re-dicta cada vez que haga falta, por WhatsApp, en modo jefe.
Memoria de "pedido reciente" (si el mismo cliente escribe de nuevo)	**30 minutos** desde que se despachó el pedido	Si un cliente escribe otra vez a los 10 minutos para agregar algo, el bot se acuerda de que acaba de pedir y no lo trata como si fuera la primera vez. Pasados los 30 minutos, se olvida solo — ya no tiene sentido "recordarlo" para ese propósito.
Historial de la conversación (los mensajes de ida y vuelta)	Se borra **apenas se cierra un pedido** (la próxima conversación arranca en blanco) — o,	Un cliente que saluda dos horas después de un pedido viejo no necesita que el bot le siga el hilo de esa conversación pasada; y una vez que un pedido ya se

Qué	Cuánto dura	Por qué se resetea así
	si el cliente nunca cerró nada, **a las 6 horas de inactividad**	despachó, seguir arrastrando esos mensajes solo gasta contexto y puede confundir al próximo pedido.
Mensajes ya vistos de Meta (para no duplicar una comanda si WhatsApp reenvía el mismo mensaje)	**7 días**	Es una tabla puramente técnica anti-duplicados, no "memoria" en el sentido de aprendizaje — se poda sola para que no crezca sin límite.
Alertas fallidas / pedidos rechazados / esperando cobro o comprobante	Hasta que alguien las resuelve o las descarta desde el panel del POS	Tampoco es aprendizaje: son colas de trabajo pendiente (avisos que no llegaron, pedidos que necesitan un dato), no conocimiento que el bot use para decidir mejor en el futuro.

En resumen: nada de esto hace que el bot "sepa más" con el tiempo — es solo para que no arrastre contexto viejo o vencido dentro de una misma jornada.

---

3. Lo que PARECE aprendizaje pero es solo una consulta en vivo

Hay una pieza que puede confundirse con "el bot se acuerda de mí": cuando un cliente escribe por primera vez en una conversación nueva, el bot le pregunta al POS (`GET /api/clientes/...`) si ese teléfono ya es un cliente conocido, y si tiene nombre o dirección guardados, evita repreguntar de cero ("¿el de siempre, a tu dirección de siempre?").

Esto NO es memoria del bot. El bot no guarda nada de esa consulta — pregunta de nuevo cada vez que arranca una conversación. Lo que de verdad "recuerda" al cliente es la base de datos del **POS** (`clientes` / `direcciones_cliente`), que se arma con cada pedido real que pasa por el cajero — esa sí es una base que crece con el tiempo, pero es la memoria del POS, no del bot. Si el POS está caído o inalcanzable en ese momento, el bot simplemente no recibe la nota y pregunta todo de cero (con un tiempo de espera corto, 2 segundos, para no hacer esperar al cliente por esto).

Guía rápida: "quiero que el bot cambie tal cosa"

- **"Que sepa que los sábados cerramos antes"** → Christian edita `conocimiento_local.md`.
- **"Que hoy no hay delivery" / "se acabó tal producto"** → Christian se lo dice al bot por WhatsApp (modo jefe) — se resetea solo a la medianoche, no hace falta acordarse de sacarlo.
- **"Que el bot sea más cálido / más breve / que nunca prometa tal cosa"** → pedirle a Alma que ajuste `system_prompt.md` en una sesión.
- **"Que reconozca esto como un patrón distinto en la conversación / cambie una regla de negocio (ej. límite de aderezos, cómo cobra el delivery)"** → eso es código: sesión con Alma, se prueba, se commitea, se pushea, se aprueba el deploy en Railway.
- **"Que se acuerde de un cliente para siempre"** → eso ya lo hace el POS (no el bot) cada vez que un pedido pasa por el cajero — no hace falta tocar nada para eso.
- **"Que cambie un precio, agregue un producto o arme una promo nueva"** → Christian lo edita desde ⚙ Admin → Menú, en el POS. El bot se entera solo, en máximo 5 minutos.